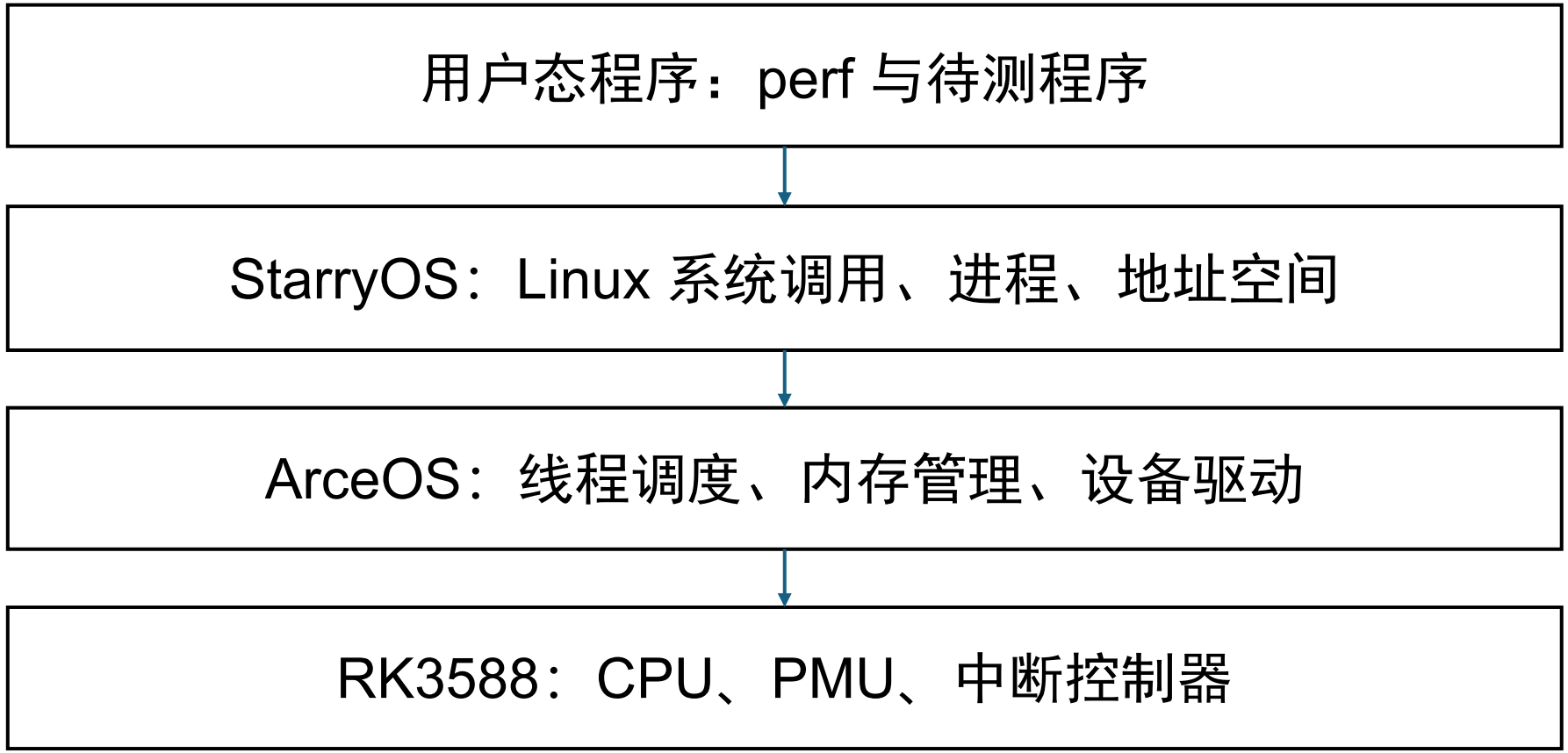
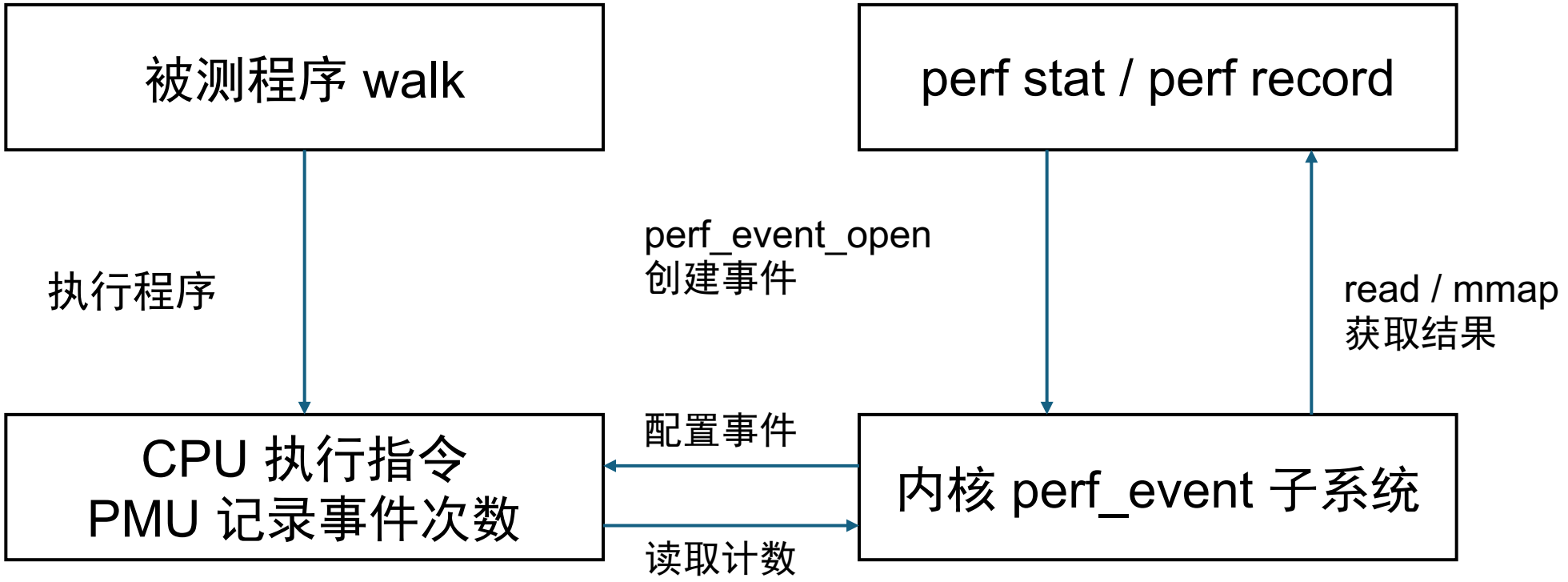
StarryOS 的 perf 支持

StarryOS 与 ArceOS



测量流程


被测程序 walk
perf stat / perf record
执行程序
perf_event_open
创建事件件
read / mmap
获取结果
CPU 执行指令
PMU 记录事件次数
配置事件
读取计数
内核 perf_event 子系统

访问顺序改变缓存行为

```
#define N (64u << 20)
static uint32_t a[N];

__attribute__((noipa))
static uint64_t walk(unsigned stride)
{
    uint64_t sum = 0;
    for (unsigned offset = 0; offset < stride; ++offset) {
        for (unsigned i = offset; i < N; i += stride)
            sum += a[i];
    }
    return sum;
}
```

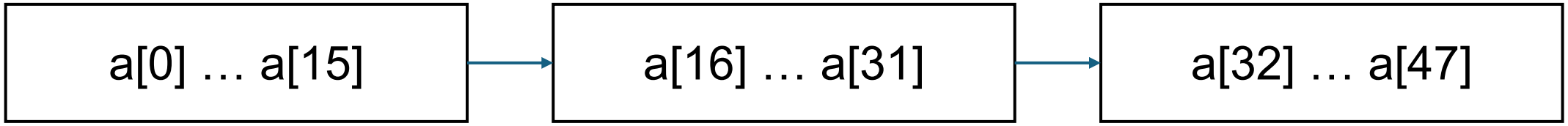
256 MiB 数组，每个元素占 4 B
最后一个参数 8 表示遍历轮数

```
./walk 1 8
```

```
./walk 16 8
```

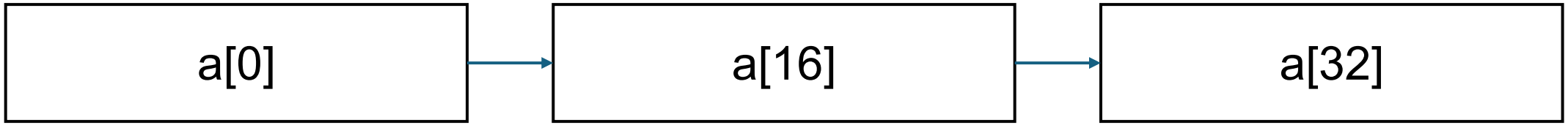
访问顺序与缓存行复用

stride = 1：依次读取同一缓存行中的 16 个元素



每个方框表示一条 64 B 缓存行，可容纳 16 个 uint32_t 元素

stride = 16：本轮从每条缓存行读取一个元素



下一轮 offset = 1：读取 a[1]、a[17]、a[33]，继续遍历整个数组

perf stat 的计数结果

```
sudo perf stat -r 5 \
    -e '{cycles:u,instructions:u,cache-misses:u}' \
    taskset -c 0 ./walk 16 8
```

指标	stride = 1	stride = 16
指令数	3,557,062,375	3,557,063,931
周期数	893,555,122	6,657,819,890
cache-misses	7,528,721	424,897,797
耗时	0.39843 秒	2.6306 秒

Xeon Gold 5318Y, Linux 6.12, CPU 0，包括初始化及 8 轮遍历，取 5 次平均

每个周期完成的指令数

指标	stride = 1	stride = 16
指令数	3,557,062,375	3,557,063,931
周期数	893,555,122	6,657,819,890
IPC	3.98	0.53

```
IPC = instructions ÷ cycles
3,557,062,375 ÷ 893,555,122 = 3.98
3,557,063,931 ÷ 6,657,819,890 = 0.53
```

perf record 采集执行位置

```
sudo perf record -e cycles:u -F 499 -g --sample-cpu \
    -o out/perf.data \
    taskset -c 0 ./walk 16 8
```

```
[ perf record: Woken up 2 times to write data ]
[ perf record: Captured and wrote 0.146 MB out/perf.data (1340 samples) ]
```

`-e cycles:u`	统计用户态周期
`-F 499`	每秒采样约 499 次
`-g`	记录调用关系
`--sample-cpu`	记录 CPU 编号

采样记录中的执行信息

```
perf script -i out/perf.data \
    -F comm,tid,cpu,time,period,ip,sym
```

```
walk 1908981 [000] 1177047.743723: 919167
    56245fa332a6 initialize
    7f071d184ca8 [unknown]
```

1908981	线程编号
1177047.743723	采样时间，单位为秒
919167	period，单位为用户态周期
56245fa332a6	IP，位于 initialize

perf report 按函数累计周期

```
perf report -i out/perf.data --stdio --no-children \
    --sort dso,sym -g none -n --show-total-period
```

```
# Overhead  Samples        Period  Symbol
    98.79%     1294    6662439286  walk
     1.21%       46      81561406  initialize


# Event count (approx.): 6744000692
# Total Lost Samples: 0
```

从样本计算函数占比

函数	累计 period	估算的周期占比
walk	6,662,439,286	98.79%
initialize	81,561,406	1.21%
合计	6,744,000,692	100%

walk 的估算周期占比

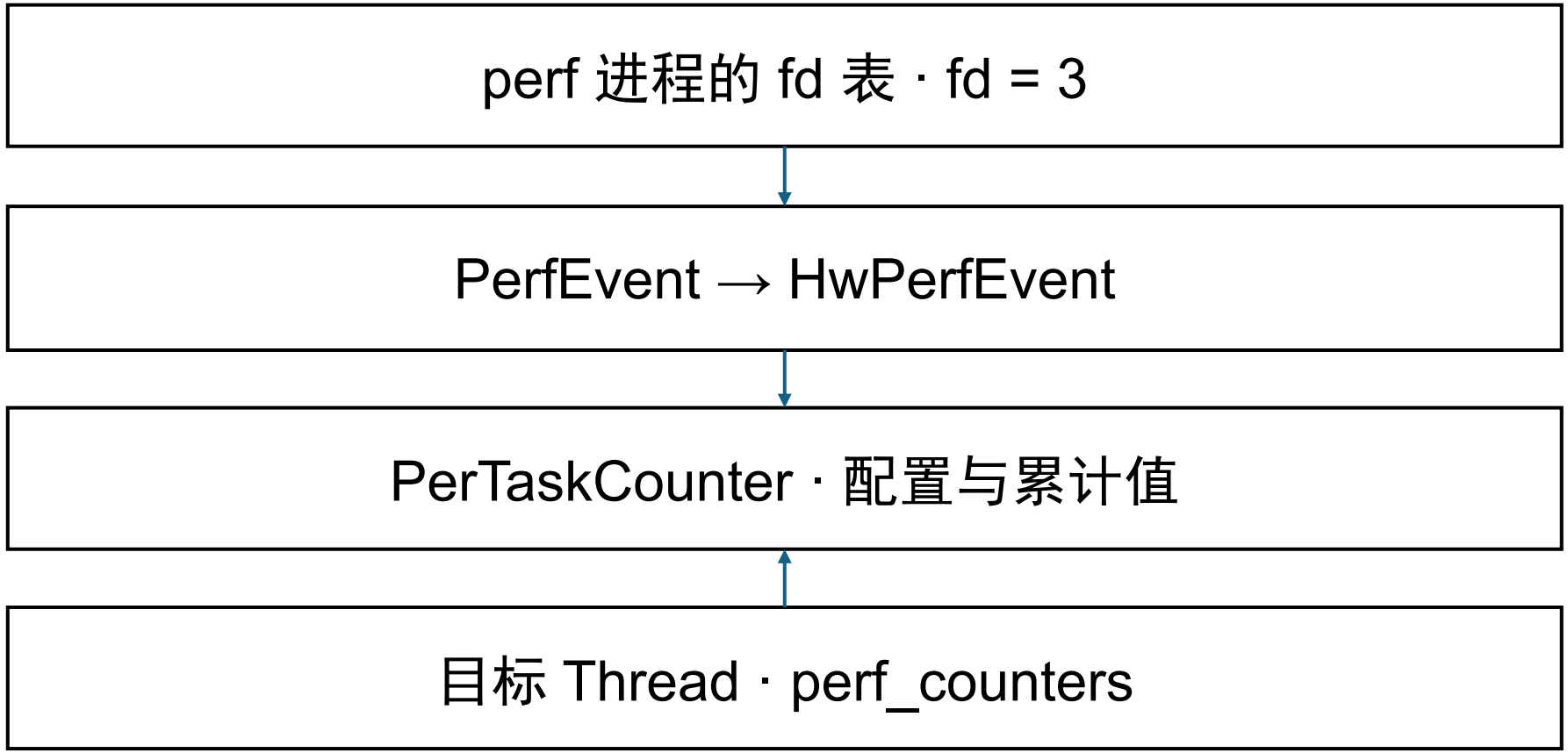
6,662,439,286 ÷ 6,744,000,692 × 100% = 98.79%

perf_event_open 创建事件文件

```
perf_event_open({
    type=PERF_TYPE_HARDWARE,
    config=PERF_COUNT_HW_CPU_CYCLES,
    disabled=1, exclude_kernel=1,
    enable_on_exec=1, ...
}, 1910302, -1, -1, PERF_FLAG_FD_CLOEXEC) = 3
```

pid = 1910302	被测线程编号
cpu = -1	随被测线程迁移
group_fd = -1	创建独立事件
fd = 3	perf 进程持有的文件描述符

事件文件与线程共享计数对象



事件文件和线程分别保存引用

创建事件时，将同一个 PerTaskCounter 交给线程和事件文件

```
super::task::attach(thr, ptc.clone());
// HwPerfEvent 的字段初始化（节选）
per_task: Some(ptc),
```

线程列表持有同一个 PerTaskCounter 的强引用

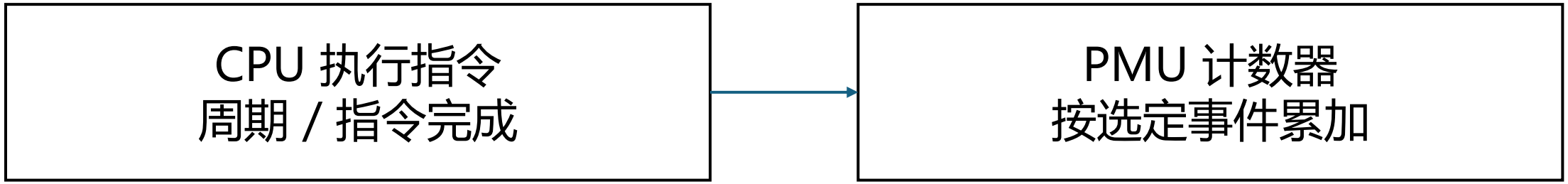
```
pub fn attach(thr: &Thread, ptc: Arc<PerTaskCounter>) {
    thr.perf_counters.lock().push(ptc);
    // 省略活跃事件数量的更新
}
```

线程切换时保存和恢复计数

```
if let Some(ext) = prev_task.task_ext() {
    ext.on_leave()
}
if let Some(ext) = next_task.task_ext() {
    ext.on_enter()
}
```

on_leave → perf_sched_out：保存计数，停用计数器
on_enter → perf_sched_in：分配计数器，恢复测量

RK3588 的 ARM PMU 计数器



Linux config = PERF_COUNT_HW_CPU_CYCLES（0）
ARM PMU 事件编码 = 0x11，选择 CPU 周期
ptc.event 保存硬件事件编码
计数器编号 n 选择 0–5 号计数器

线程调入时配置计数器

```
ax_cpu::pmu::counter::configure(
    n,
    ptc.event,
    ptc.exclude_user,
    ptc.exclude_kernel,
);
ax_cpu::pmu::counter::enable(n);
```

on_enter → perf_sched_in → arm_slice
配置事件与过滤条件、清零，然后启用计数

用户态与内核态的事件过滤

```
if exclude_el0 {
    evtyper |= EVTYPER_U_EXCLUDE_EL0;
}
if exclude_el1 {
    evtyper |= EVTYPER_P_EXCLUDE_EL1;
}
write_typer(n, evtyper);
reset(n);
```

U[30] 排除用户态

P[31] 排除内核态

同一段 2000 万次计算循环，统计 INST_RETIRED

用户态：exclude_user=0，exclude_kernel=1　160,000,040

内核态：exclude_user=1，exclude_kernel=0　35,034

线程 A 开始运行

CPU 0

CPU 的硬件计数器	A 的软件累计值	B 的软件累计值
H = 0 当前统计线程 A	A.total = 0 保存在 A 的事件对象	B.total = 0 保存在 B 的事件对象

```
ax_cpu::pmu::counter::configure(
    n, ptc.event, ptc.exclude_user, ptc.exclude_kernel);
ax_cpu::pmu::counter::enable(n);
```

调出 A 时保存计数

CPU 0	CPU 的硬件计数器	A 的软件累计值	B 的软件累计值
	H = 120 读出后停用计数器	A.total = 120 0 + 120	B.total = 0 尚未运行

```
let delta = ax_cpu::pmu::counter::read(n);
if accumulate {
    ptc.accumulated.fetch_add(delta, Ordering::AcqRel);
}
ax_cpu::pmu::counter::disable(n);
```

线程 B 使用同一计数器

CPU 0

CPU 的硬件计数器	A 的软件累计值	B 的软件累计值
H = 50 当前统计线程 B	A.total = 120 保留 A 的测量结果	B.total = 0 调出 B 时才保存 50

```
ax_cpu::pmu::counter::configure(
    n, ptc.event, ptc.exclude_user, ptc.exclude_kernel);
ax_cpu::pmu::counter::enable(n);
```

线程 A 再次开始运行

CPU 0

CPU 的硬件计数器	A 的软件累计值	B 的软件累计值
H = 0 当前统计线程 A	A.total = 120 保留前次运行的计数	B.total = 50 B 已经调出

```
ax_cpu::pmu::counter::configure(
    n, ptc.event, ptc.exclude_user, ptc.exclude_kernel);
ax_cpu::pmu::counter::enable(n);
```

读取历史计数与运行中的增量

```
let mut value = ptc.accumulated.load(Ordering::Acquire);
if ptc.running.load(Ordering::Acquire)
    && ptc.last_cpu.load(Ordering::Acquire)
        == ax_hal::percpu::this_cpu_id()
{
    let n = ptc.slot.load(Ordering::Acquire);
    if n != NO_SLOT {
        value += ax_cpu::pmu::counter::read(n);
    }
}
```

A 自己读取时，同一 CPU 返回 120 + 30 = 150
A 在 CPU 0 运行时，CPU 1 的读取路径只取得已保存的 120
中途 read 保持 accumulated = 120，调出时累计本次运行的计数

再次调出 A 时累计计数

CPU 0

CPU 的硬件计数器	A 的软件累计值	B 的软件累计值
H = 80 读出后停用计数器	A.total = 200 120 + 80	B.total = 50 B 已经调出

```
let delta = ax_cpu::pmu::counter::read(n);
if accumulate {
    ptc.accumulated.fetch_add(delta, Ordering::AcqRel);
}
ax_cpu::pmu::counter::disable(n);
```

独立事件的轮换

事件	0–10 ms	10–20 ms
E0	C0 计数 100	尚未开始
E1	C1 计数 60	尚未开始
E2	等待 计数 0	尚未开始

同一线程启用 E0、E1、E2，硬件只有 C0、C1 这 2 个计数器
time_enabled 每个事件均为 10 ms
time_running E0、E1 占用硬件 10 ms，E2 为 0 ms

下一轮分配计数器

事件	0–10 ms	10–20 ms
E0	C0 计数 100	等待 计数 0
E1	C1 计数 60	C1 计数 60
E2	等待 计数 0	C0 计数 40

3 个事件的 time_enabled 均为 20 ms
time_running E0 为 10 ms，E1 为 20 ms，E2 为 10 ms
累计计数 E0 为 100，E1 为 120，E2 为 40

按实际计数时间估算

事件	累计计数	缩放计算
E0	100	100 × 20/10 = 200
E1	120	120 × 20/20 = 120
E2	40	40 × 20/10 = 80

估算计数 = 累计计数 × time_enabled / time_running
计数期间的程序行为需要能够代表整个启用期间

定时器推动事件轮换

```
if !in_window && ptc.running.load(Ordering::Acquire) {
    disarm_slice(ptc, now, true);
}
// 第二次遍历：为本轮事件分配计数器（节选）
if in_window && !ptc.running.load(Ordering::Acquire)
    && let Some(n) = super::percpu::alloc_programmable_counter()
{
    arm_slice(ptc, n, now, thr.proc_data.proc.pid() as u32, thr.tid());
}
```

in_window 表示事件被选入本轮，例如 E1、E2
第一次遍历停止 E0，保存计数与时间，并释放计数器
第二次遍历为 E2 分配计数器并启用；E1 保持运行

采样计数器的预置值

计数器 H = 236	溢出标志 0	已保存的样本 0

```rust
pub fn preload(n: usize, period: u32) {
    write(n, (0u32).wrapping_sub(period) as u64);
}
```

period = 20，连续计数增加 20 次后请求中断
8 位初值为 256 − 20 = 236；32 位初值为 2^{32} − period
本例先假设线程持续运行，直到触发一次采样

19 次事件后的计数值

计数器	溢出标志	已保存的样本
H = 255	0	0

236 + 19 = 255
硬件每发生一次所选事件，计数器递增
此时计数器尚未回绕

第 20 次事件触发溢出

计数器	溢出标志	已保存的样本
H = 0	1	0

255 + 1 按 8 位回绕为 0
硬件设置溢出标志，并触发 PMU 中断
CPU 保存被中断程序的现场

中断处理后重新预置

计数器 H = 236	溢出标志 0	已保存的样本 1

构造并写入采样记录
调用 preload 设置下一次采样间隔
清除 PMU 溢出标志，随后继续执行程序

线程切换后的采样间隔

```
period = 20，线程 A 已计数 19 次，硬件值为 255
调出 A 时，停止计数并撤销采样登记
再次调入 A 时，下面的代码重设完整间隔
```

```
ax_cpu::pmu::counter::preload(n, ptc.sample_period);
// 每次调入都使用完整的 sample_period
```

```
重设 236 后，还需 20 次事件才采样
要延续采样间隔，需要保存剩余的 1 次，下次预置 255
```

PMU 中断入口取得指令地址

```
let ip = ax_cpu::pmu::interrupted_pc();
let is_user = ax_cpu::pmu::interrupted_is_user();

let ovf = ax_cpu::pmu::overflow::status();
```

AArch64 进入异常时，ELR_EL1 保存异常返回地址

```
pub fn interrupted_pc() -> u64 {
    let value;
    unsafe {
        asm!("mrs {}, ELR_EL1", out(reg) value);
    }
    value
}
```

按计数器编号查找采样事件

```
for n in 0..=MAX_COUNTER {
    if ovf & (1 << n) == 0 {
        continue;
    }
    handled |= 1 << n;
```

```
    let registry = unsafe { REGISTRY.current_ref_mut_raw() };
    let Some(slot) = registry[n].as_mut() else {
        continue;
    };

    let sample_type = slot.sample_type;
    let ring_vaddr = slot.ring_vaddr;
    let ring_len = slot.ring_len;
```

Linux 上采集的 80 B 样本

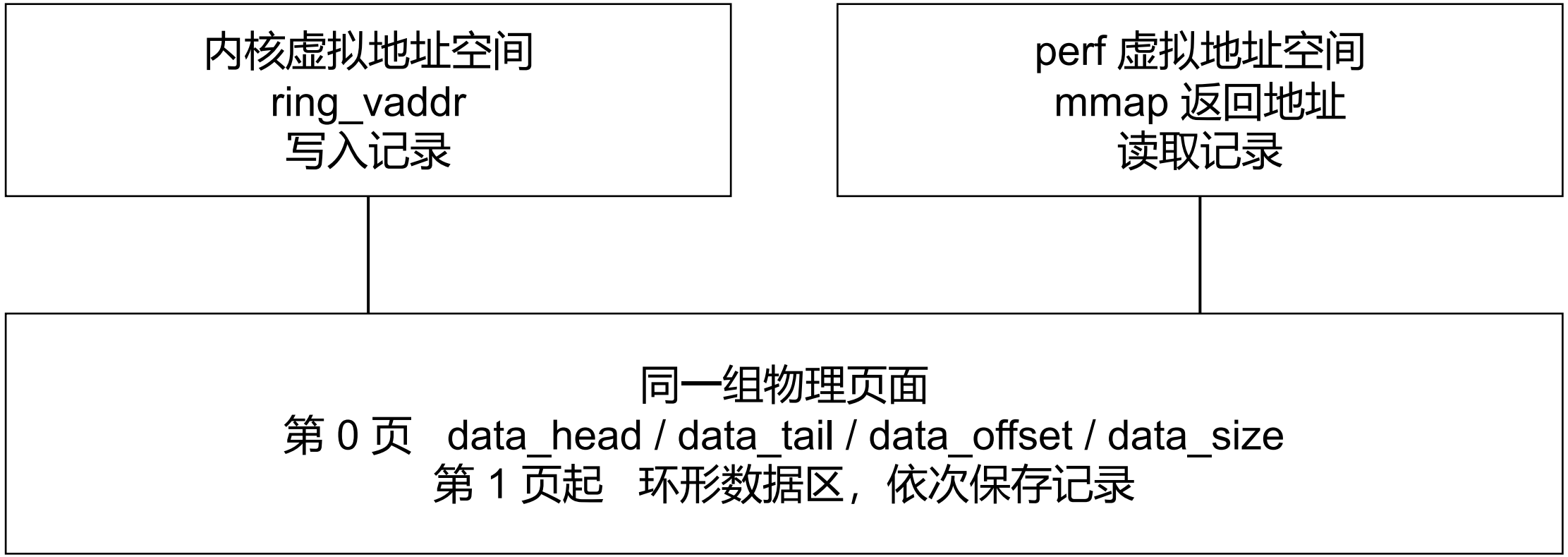
偏移（B)	字段与长度	实际取值
0	HEADER 8 B	type 9 · misc 2 · size 80
8	IP 8 B	0x56245fa332a6
16	PID / TID 8 B	1908981 / 1908981
24	TIME 8 B	117704774372360 0 ns
32	CPU / 保留字段 8 B	0 / 0
40	PERIOD 8 B	919167
48	CALLCHAIN 32 B	nr = 3 0xffffffffffffe00 0x56245fa332a6 0x7f071d184ca8

sample_type 选择记录字段

```
if sample_type & PERF_SAMPLE_IP != 0 {
    put!(d.ip);
}
if sample_type & PERF_SAMPLE_TID != 0 {
    put!(d.pid);
    put!(d.tid);
}
if sample_type & PERF_SAMPLE_TIME != 0 {
    put!(d.time);
}
```

去掉 TIME 的 8 B：CPU 偏移由 32 变为 24，总长度为 72 B
PERIOD 原来位于偏移 40，现在应位于哪里？

内核与 perf 共享同一组物理页



写入前检查环形缓冲区容量

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
						X0	X1	X2	X3	X4	X5	X6	X7		

head = 14　tail = 6　待写入 A B C D E F（6 B）
X0–X7 为未读字节；A–F 为新增字节，用于演示位置变化

```
if head.wrapping_sub(tail).wrapping_add(len as u64)
    > data_size as u64 {
    return false;
}
// 14 - 6 + 6 = 14 <= 16, 可以写入
```

第一段复制到数据区末尾

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
						X0	X1	X2	X3	X4	X5	X6	X7	A	B

累计 head = 14，tail = 6
A、B 写入偏移 14、15，新增内容尚未发布

```
let start = (head % data_size as u64) as usize;
let first = core::cmp::min(len, data_size - start);
core::ptr::copy_nonoverlapping(
    record.as_ptr(), (data_base + start) as *mut u8, first);
```

第二段回到数据区开头

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C	D	E	F			X0	X1	X2	X3	X4	X5	X6	X7	A	B

累计 head = 14，tail = 6
C–F 写入偏移 0–3，全部字节已复制，新增内容尚未发布

```
if first < len {
    core::ptr::copy_nonoverlapping(
        record.as_ptr().add(first),
        data_base as *mut u8,
        len - first,
    );
}
```

发布写入位置 head = 20

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C	D	E	F			X0	X1	X2	X3	X4	X5	X6	X7	A	B

内核先写完内容，再公布 head = 20
perf 取得新 head 后，读取累计位置 6–20 的 14 B

```
// Linux 内核写入端
smp_wmb();
WRITE_ONCE(rb->user_page->data_head, head);

// Linux perf 读取端（AArch64）
return smp_load_acquire(&base->data_head);
```

跨边界读取数据

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C	D	E	F			X0	X1	X2	X3	X4	X5	X6	X7	A	B

末尾 10 B 与开头 4 B，分别调用 push 保存
下面节选地址与长度计算，start 表示累计读取位置

```
// 末尾一段，start = 6，mask = 15（Linux 代码节选）
buf = &data[md->core.start & md->core.mask];
size = md->core.mask + 1 - (md->core.start & md->core.mask);
md->core.start += size;
// 开头一段，start = 16，end = 20
buf = &data[md->core.start & md->core.mask];
size = md->core.end - md->core.start;
md->core.start += size;
```

读取完成后归还空间

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
C	D	E	F			X0	X1	X2	X3	X4	X5	X6	X7	A	B

两段共 14 B 已保存，tail 从 6 更新为 20
内核观察到新 tail 后，才可以复用这些位置

```
// perf record 保存成功后
md->core.prev = head;
perf_mmap__consume(&md->core);

// Linux perf 的 AArch64 发布操作
smp_store_release(&base->data_tail, tail);
```

采样记录的写入与丢失

```
let len = build_sample(&mut record[..], sample_type, misc, &data);
let wrote = unsafe {
    ring_write(ring_vaddr, ring_len, &record[..len])
};
```

```
if !wrote && !lost_ptr.is_null() {
    unsafe {
        (*(lost_ptr as *const AtomicU64))
            .fetch_add(1, Ordering::Relaxed)
    };
}

ax_cpu::pmu::counter::preload(n, next_period);
```

共享页面为什么读到 0

RK3588 采样没有输出，用户侧 data_head 一直为 0
同时读取两侧 data_size 和动态 data_head
data_size 由初始化写入，data_head 随记录写入而变化

内核映射 Normal / Write-Back
用户映射 Physical（UNCACHED）改为 PhysicalCached

data_size 正常只说明初始化值可见
继续比较两侧 head、物理页地址及映射属性

固定间隔采样的板级检查

```
a.type = PERF_TYPE_RAW; // ARM 事件编码
a.config = EV_CPU_CYCLES;
a.sample_period = 1000000;
a.sample_type = SAMPLE_IP; // 记录 IP 字段
```

```
SAMP-1 PASS samples=2048 in_loop_range=2022
```

2 亿次循环，停用后读取，要求至少 8 条
地址观察范围：busy 至 busy+4096，不参与 PASS 判定
8 × 4096 B ÷ 每条 16 B = 2048 条，恰好达到容量

采样事件组的创建

```
a.sample_period = 100000ull;
a.sample_type = SAMPLE_IP | SAMPLE_TID |
                SAMPLE_TIME | SAMPLE_READ;
a.read_format = FORMAT_GROUP | FORMAT_ID;
long leader = peo(&a, self, -1, -1, 0);

m.sample_period = 0;
m.read_format = FORMAT_GROUP | FORMAT_ID;
long member = peo(&m, self, -1, leader, 0);
```

本测试用两个周期事件，检查组采样的记录格式
SAMPLE_READ 附带计数；FORMAT_GROUP 返回整组结果；FORMAT_ID 附加事件编号
组长连续计数 100000 个周期后请求采样，记录中附带整组计数

组采样记录的 72 B 布局

偏移（B）	字段与长度	内容
0	HEADER 8 B	type 9 · size 72
8	IP 8 B	采样时的指令地址
16	PID / TID 8 B	目标进程与线程
24	TIME 8 B	采样时间
32	nr 8 B	2 个事件
40	value / ID 16 B	已处理溢出的 period 合计、ID
56	value / ID 16 B	成员历史值 + 硬件增量、ID

SampleSlot 保存成员计数的地址

```
let Some(m) = weak.upgrade() else {
    continue;
};
out[n] = sampling::GroupMember {
    id: m.sample_id.load(Ordering::Relaxed),
    accumulated: &m.accumulated as *const AtomicU64 as *const (),
    slot: &m.slot as *const AtomicUsize as *const (),
    last_cpu: &m.last_cpu as *const AtomicUsize as *const (),
    running: &m.running as *const AtomicBool as *const (),
};
```

准备采样时 m 是临时强引用，登记表只保存字段地址
中断中 m 为登记信息，acc 为历史值，当前硬件增量另行加入

```
let acc = (*(m.accumulated as *const AtomicU64))
    .load(Ordering::Relaxed);
```

跨线程成员释放后的采样引用

事件组

线程 A	SampleSlot	线程 B：成员 M
拥有组长 L 继续运行采样事件	仍保存 M 的计数地址 中断将读取该地址	成员线程对象销毁 perf 关闭成员 fd 最后一个 Arc 释放

组长用 Weak 保存成员，M 的全部 Arc 释放后对象销毁
SampleSlot 仍保存 M 的原始地址，中断继续读取该地址

组长与成员属于同一线程

事件组	线程 A	SampleSlot	线程 B
	持有组长 L 的 Arc 持有成员 M 的 Arc	保存 M 的计数地址 中断读取 M 的计数	没有组内事件

调出时撤销登记；列表引用保留到线程对象销毁

```
if ptc.is_sampling {
    ax_cpu::pmu::counter::disable(n);
    ax_cpu::pmu::overflow::disable_irq(n);
    sampling::unregister(n);
}
```

事件组成员的线程归属

```
if leader.owner_tid != member.owner_tid {
    return Err(AxError::InvalidInput);
}
```

组长 TID 41，成员 TID 52　返回 EINVAL
组长 TID 41，成员 TID 41　继续创建成员

事件组的回归测试

```
long xfd = perf_event_open(&mattr, s.b_tid, -1, leader, 0ul);
int xerr = errno;
if (xfd >= 0) {
    rc = 1;
} else if (xerr != EINVAL) {
    rc = 1;
}

long sfd = perf_event_open(&sattr, self_tid, -1, leader, 0ul);
if (sfd < 0) {
    rc = 1;
}
```

GRP-2 PASS cross-thread member EINVAL;
same-thread member opens (UAF gate holds)

组采样的板级检查

```
int ok = !wr.failed && wr.samples >= 2 &&
         wr.bad_nr == 0 && wr.bad_id == 0 &&
         wr.zero_memb == 0;
```

```
SREAD-2 PASS group-sample samples=244 nr==2 ids-match
```

负载读取 /dev/zero，每次 4096 B，循环上限 400000 次
负载期间启用事件组，停用后解析共享缓冲区
检查已解析样本的 nr = 2、事件 ID 与成员计数非零